```
#include <atomic>
#include <thread>
#include <assert.h>

std::atomic<bool> x,y;
std::atomic<int> z;

void write_x_then_y()
{
    x.store(true,std::memory_order_relaxed);
    y.store(true,std::memory_order_release);
}

void read_y_then_x()
{
    while(!y.load(std::memory_order_acquire));
    if(x.load(std::memory_order_relaxed))
        ++z;
}

int main()
{
    x=false;
    y=false;
    z=0;
    std::thread a(write_x_then_y);
    std::thread b(read_y_then_x);
    a.join();
    b.join();
    assert(z.load()!=0);
}
```

❶ Spin, waiting for y to be set to true

❷

❸

❹

❺

Eventually, the load from `y` ❸ will see `true` as written by the store ❷. Because the store uses `memory_order_release` and the load uses `memory_order_acquire`, the store synchronizes-with the load. The store to `x` ❶ happens-before the store to `y` ❷, because they're in the same thread. Because the store to `y` synchronizes-with the load from `y`, the store to `x` also happens-before the load from `y` and by extension happens-before the load from `x` ❹. Thus the load from `x` *must* read `true`, and the assert ❺ *can't* fire. If the load from `y` wasn't in a `while` loop, this wouldn't necessarily be the case; the load from `y` might read `false`, in which case there'd be no requirement on the value read from `x`. In order to provide any synchronization, acquire and release operations must be paired up. The value stored by a release operation must be seen by an acquire operation for either to have any effect. If either the store at ❷ or the load at ❸ was a relaxed operation, there'd be no ordering on the accesses to `x`, so there'd be no guarantee that the load at ❹ would read `true`, and the `assert` could fire.

You can still think about acquire-release ordering in terms of our men with notepads in their cubicles, but you have to add more to the model. First, imagine that every store that's done is part of some batch of updates, so when you call a man to tell him to write down a number, you also tell him which batch this update is part of: "Please write down 99, as part of batch 423." For the last store in a batch, you tell this to the man too: "Please write down 147, which is the last store in batch 423." The man in the cubicle will then duly write down this information, along with who gave him the value. This models a store-release operation. The next time you tell